

INFORME TÉCNICO – Casos de Uso UC-07, UC-12 y UC-14 y Desarrollo Realizado
DashKPIs

Estudiante:

Matias Esteban Marin Chacón

Equipo:

Print (Null);

Universidad Piloto de Colombia

Docente:

Gilberto Pedraza

Contenido	
Introducción	2
Descripción General	2
UC-07 – Ver Notificaciones	2
UC-12 – Exportar Reportes en PDF	2
UC-14 – Aplicar Filtros en Reportes	3
Relación Técnica entre los Casos de Uso	3
Desarrollo Realizado	3
Defectos Identificados durante el desarrollo	4
Aspectos técnicos críticos	4
Conclusiones	5

Introducción

Este informe técnico presenta un análisis integrado de los casos de uso **UC-07 (Ver Notificaciones)**, **UC-12 (Exportar Reportes en PDF)** y **UC-14 (Aplicar Filtros en Reportes)** del proyecto DASHKPIs, junto con un resumen del desarrollo real ejecutado.

El objetivo es describir cómo interactúan los módulos, los desafíos técnicos enfrentados y los defectos identificados durante el ciclo de trabajo.

Descripción General

UC-07 – Ver Notificaciones

Facilita la visualización de notificaciones generadas por el sistema, tales como alertas, mensajes administrativos y confirmaciones de procesos. Permite revisar el contenido detallado de cada notificación, su estado (leída/no leída) y mantener un historial sincronizado.

UC-12 – Exportar Reportes en PDF

Permite al usuario generar un archivo PDF basado en el reporte visible en pantalla. La exportación debe conservar el formato, estilo, filtros aplicados y pie de página institucional. Requiere coordinación entre el módulo de presentación y el servicio backend de generación de documentos PDF.

UC-14 – Aplicar Filtros en Reportes

Este caso de uso permite al usuario aplicar criterios de filtrado para obtener reportes segmentados según fechas, usuarios, KPIs, estados o categorías. La funcionalidad requiere validación de parámetros, manejo correcto de rangos de fechas, actualización dinámica de la vista y comunicación constante entre frontend y backend para generar la información filtrada.

Relación Técnica entre los Casos de Uso

Los tres casos de uso están conectados funcional y técnicamente:

- **UC-14 → UC-12:** Los filtros aplicados modifican los datos exportados en el PDF. Si los filtros fallan, la exportación puede resultar en documentos incompletos o vacíos.
- **UC-12 → UC-07:** Al completar la generación del PDF, el sistema debe producir una notificación para el usuario. Si la exportación falla, el módulo de notificaciones reflejará errores o ausencia de alertas.
- **UC-07 ← UC-14:** Aplicar filtros también puede generar notificaciones internas o mensajes de estado, los cuales deben visualizarse correctamente en UC-07.

Estos casos de uso dependen de una correcta interacción entre frontend, backend, estado global de la aplicación y gestión de servicios externos.

Desarrollo Realizado

Durante el desarrollo se implementaron los siguientes componentes:

- **Frontend (React):**
 - Vista de filtros y menú desplegable de opciones.
 - Conexión al backend mediante Fetch/Axios.
 - Renderización dinámica de reportes filtrados.
 - Componente de exportación PDF.
 - Panel de notificaciones con estados leída/no leída.
- **Backend:**
 - Endpoint para filtrar reportes.
 - Endpoint para exportación PDF.
 - Endpoint de obtención de notificaciones.

- Validación de parámetros y manejo de rangos de fecha.
- Configuración de CORS.
- Integración con servicio generador de PDF.
- **Gestión del estado:**
 - Hooks useState y useEffect.
 - Control de eventos y recarga de datos.
 - Manejo de errores para respuestas vacías o fallas del servidor.

Defectos Identificados durante el desarrollo

Durante el proceso se detectaron múltiples defectos relevantes:

1. El frontend no mostraba los datos recibidos del backend.
Los componentes no estaban renderizando los resultados por errores en props y estados.
2. Pérdida total del módulo frontend debido a un error de merge.
Se sobrescribió el avance del módulo, obligando a reconstruirlo desde cero.
3. Los filtros no aplicaban correctamente rangos de fecha.
El backend interpretaba incorrectamente los límites, generando resultados vacíos.
4. El módulo de exportación PDF generaba archivos vacíos.
La estructura enviada al backend no incluía los datos del reporte.
5. El panel de notificaciones no cargaba.
Existía una inconsistencia entre los endpoints del frontend y el backend.
6. El sistema no mostraba mensaje cuando no había datos filtrados.
Faltaba la validación y visualización del estado sin resultados.
7. Errores de CORS en la exportación PDF.
El backend no permitía el acceso del frontend al recurso.
8. Los selectores de filtros no cargaban opciones.
useEffect tenía dependencias incorrectas y nunca ejecutaba la llamada inicial.

Estos defectos fueron corregidos en fases de pruebas funcionales y de integración, documentados en el LOGDEFECTOS del ciclo.

Aspectos técnicos críticos

- Falta de sincronización de estados en React generó fallos en renderizado.

- Validaciones incompletas de datos en frontend y backend produjeron respuestas vacías.
- Las rutas inconsistentes entre front y back provocaron errores 404 en notificaciones.
- Problemas de configuración CORS bloquearon funcionalidades críticas.
- Un error de gestión de repositorio ocasionó la pérdida total de un módulo.
- El manejo de errores no contemplaba situaciones como reportes vacíos, filtros inválidos o fallos del servidor.

Conclusiones

Los casos de uso analizados presentan alta interdependencia técnica, especialmente en torno al flujo de filtrado → exportación → notificaciones. Los defectos encontrados reflejan problemas comunes en sistemas con componentes altamente sincronizados.

El desarrollo permitió identificar y corregir aspectos clave como validación de datos, manejo de estado, rutas de API y configuraciones del servidor. La reconstrucción del módulo frontend evidenció la importancia de una adecuada gestión del repositorio y control de versiones.